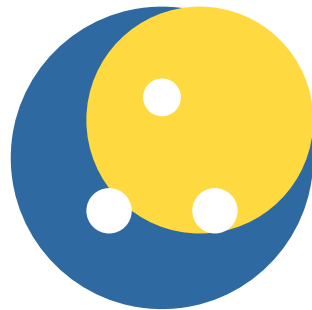


Python para Análisis de Datos

Guía Profesional Completa

Edición 2026



Informe Técnico Especializado

Departamento de Ciencia de Datos

Marzo 2026

Contents

1	Introducción	2
1.1	Importancia de Python en 2026	2
1.2	Ventajas Competitivas	2
2	Configuración del Entorno	2
2.1	Instalación de Python	3
2.2	Gestión de Entornos Virtuales	3
2.3	Entornos de Desarrollo	3
3	NumPy - Fundamentos Numéricos	4
3.1	Creación de Arrays	4
3.2	Propiedades de Arrays	5
3.3	Indexing y Slicing	5
3.4	Operaciones con Arrays	5
3.5	Broadcasting	6
4	Pandas - Manipulación de Datos	6
4.1	Creación de DataFrames	6
4.2	Exploración de Datos	7
4.3	Limpieza de Datos	7
4.4	Filtrado y Selección	8
4.5	GroupBy (Agrupamiento)	8
4.6	Merge, Join y Concat	9
5	Visualización de Datos	9
5.1	Matplotlib - Fundamentos	10
5.2	Seaborn - Gráficos Estadísticos	10
6	Machine Learning con Scikit-Learn	10
6.1	Preparación de Datos	11
6.2	Modelos de Clasificación	11
6.3	Validación Cruzada y Grid Search	12
7	Librerías Avanzadas	12
7.1	XGBoost	13
7.2	Plotly - Visualización Interactiva	13
7.3	Streamlit - Apps Web	14
8	Mejores Prácticas	14
8.1	Estructura de Proyecto Recomendada	15
9	Conclusiones	15
A	Recursos de Aprendizaje	16
A.1	Cursos Recomendados	16
A.2	Datasets para Practicar	16

B Referencias Bibliográficas**16**

1 Introducción

Definición

Python es un lenguaje de programación de alto nivel, interpretado y de propósito general, que se ha convertido en el estándar de facto para análisis de datos y ciencia de datos debido a su sintaxis clara y ecosistema rico de librerías especializadas.

1.1 Importancia de Python en 2026

Información Clave

Estadísticas Clave del Mercado:

- **68%** de profesionales de ML usan Python como lenguaje principal (Stack Overflow 2025)
- **8.2 millones** de desarrolladores Python en el mundo
- Salario promedio LATAM: **\$45k-85k USD/año** para ML Engineers
- Tiempo a primer empleo: **6-12 meses** con dedicación constante

1.2 Ventajas Competitivas

Table 1: Ventajas de Python para Análisis de Datos

Ventaja	Descripción
Sintaxis simple	Fácil de aprender y leer, reduce curva de aprendizaje
Ecosistema rico	Más de 200,000 librerías disponibles en PyPI
Comunidad activa	Soporte constante, actualizaciones frecuentes
Versatilidad	Desde análisis básico hasta deep learning avanzado
Gratuito	Open source sin costos de licencia
Interoperabilidad	Integración con C, C++, Java, R, SQL

2 Configuración del Entorno

2.1 Instalación de Python

Ejemplo de Código

Comandos de Instalación por Sistema Operativo:

```

1 # Ubuntu/Debian
2 sudo apt update
3 sudo apt install python3 python3-pip
4
5 # macOS (con Homebrew)
6 brew install python
7
8 # Windows
9 # Descargar instalador desde python.org

```

2.2 Gestión de Entornos Virtuales

Ejemplo de Código

Creación y Gestión de Entorno Virtual:

```

1 # Crear entorno virtual
2 python -m venv venv
3
4 # Activar entorno (Windows)
5 venv\Scripts\activate
6
7 # Activar entorno (macOS/Linux)
8 source venv/bin/activate
9
10 # Instalar paquetes principales
11 pip install pandas numpy matplotlib seaborn scikit-learn
12
13 # Guardar dependencias
14 pip freeze > requirements.txt

```

2.3 Entornos de Desarrollo

Table 2: Entornos de Desarrollo Recomendados

Herramienta	Tipo	Uso Recomendado
Jupyter Notebook	Web interactivo	Exploración, prototipado
JupyterLab	Web avanzado	Proyectos completos
VS Code	IDE	Desarrollo profesional
Google Colab	Cloud gratuito	GPU/TPU gratis, colaboración
PyCharm	IDE completo	Proyectos empresariales

3 NumPy - Fundamentos Numéricos

Definición

NumPy (Numerical Python) es la librería fundamental para cómputo numérico en Python. Proporciona arrays multidimensionales eficientes y funciones matemáticas vectorizadas.

3.1 Creación de Arrays

Ejemplo de Código

Métodos de Creación de Arrays NumPy:

```
1 import numpy as np
2
3 # Desde una lista
4 arr = np.array([1, 2, 3, 4, 5])
5
6 # Array de ceros
7 zeros = np.zeros((3, 4)) # 3 filas, 4 columnas
8
9 # Array de unos
10 unos = np.ones((2, 5))
11
12 # Array con rango
13 rango = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
14
15 # Array con valores espaciados
16 espaciado = np.linspace(0, 1, 5)
17
18 # Matriz identidad
19 identidad = np.eye(4)
20
21 # Array aleatorio
22 aleatorio = np.random.rand(3, 3)
```

3.2 Propiedades de Arrays

Table 3: Propiedades de Arrays NumPy

Propiedad	Descripción	Ejemplo
shape	Dimensiones del array	(3, 4)
ndim	Número de dimensiones	2
size	Total de elementos	12
dtype	Tipo de datos	int64
itemsize	Bytes por elemento	8

3.3 Indexing y Slicing

Ejemplo de Código

Acceso y Manipulación de Elementos:

```

1 arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
2
3 # Indexing b s i c o
4 arr[0]      # 1
5 arr[-1]     # 10
6
7 # Slicing
8 arr[2:5]    # [3, 4, 5]
9 arr[::2]    # [1, 3, 5, 7, 9]
10
11 # Array multidimensional
12 matriz = np.array([[1, 2, 3], [4, 5, 6]])
13 matriz[0, 1] # 2
14 matriz[:, 1] # [2, 5]
15
16 # Boolean indexing
17 arr[arr > 5] # [6, 7, 8, 9, 10]

```

3.4 Operaciones con Arrays

Table 4: Operaciones NumPy

Tipo	Ejemplos
Aritméticas	$a + b$, $a - b$, $a * b$, a / b , $a ** 2$
Funciones	<code>np.sqrt(a)</code> , <code>np.exp(a)</code> , <code>np.log(a)</code> , <code>np.sin(a)</code>
Estadísticas	<code>np.mean(a)</code> , <code>np.median(a)</code> , <code>np.std(a)</code> , <code>np.sum(a)</code>
Álgebra Lineal	<code>np.dot(A, B)</code> , <code>np.linalg.inv(A)</code> , <code>np.linalg.det(A)</code>

3.5 Broadcasting

Ejemplo de Código

Broadcasting - Operaciones con Diferentes Dimensiones:

```
1 # Sumar escalar a array
2 arr = np.array([1, 2, 3])
3 resultado = arr + 10 # [11, 12, 13]
4
5 # Sumar arrays de diferentes dimensiones
6 matriz = np.array([[1, 2, 3], [4, 5, 6]])
7 vector = np.array([10, 20, 30])
8 resultado = matriz + vector
9 # [[11, 22, 33], [14, 25, 36]]
```

4 Pandas - Manipulación de Datos

Definición

Pandas es la librería esencial para manipulación y análisis de datos. Proporciona estructuras de datos flexibles como Series (1D) y DataFrames (2D).

4.1 Creación de DataFrames

Ejemplo de Código

Métodos de Creación:

```
1 import pandas as pd
2
3 # Desde diccionario
4 df = pd.DataFrame({
5     'Nombre': ['Ana', 'Luis', 'Carlos'],
6     'Edad': [25, 30, 35],
7     'Salario': [50000, 60000, 55000]
8 })
9
10 # Desde CSV
11 df = pd.read_csv('datos.csv')
12
13 # Desde Excel
14 df = pd.read_excel('datos.xlsx')
15
16 # Desde URL
17 df = pd.read_csv('https://ejemplo.com/datos.csv')
```


4.2 Exploración de Datos

Table 5: Funciones de Exploración en Pandas

Función	Propósito
<code>df.head()</code>	Primeras 5 filas
<code>df.info()</code>	Tipos de datos, nulos, memoria
<code>df.describe()</code>	Estadísticas descriptivas
<code>df.shape</code>	Dimensiones (filas, columnas)
<code>df.columns</code>	Nombres de columnas
<code>df.dtypes</code>	Tipos de datos por columna
<code>df.isnull().sum()</code>	Conteo de valores nulos

4.3 Limpieza de Datos

Ejemplo de Código

Técnicas de Limpieza:

```

1  # Detectar nulos
2  df.isnull().sum()
3
4  # Eliminar nulos
5  df.dropna()
6
7  # Rellenar nulos
8  df.fillna(df.mean())
9
10 # Eliminar duplicados
11 df.drop_duplicates()
12
13 # Cambiar tipos
14 df['Edad'] = df['Edad'].astype(int)
15
16 # Renombrar columnas
17 df.rename(columns={'Nombre': 'Nombre_Completo'},
18           inplace=True)

```

4.4 Filtrado y Selección

Ejemplo de Código

Selección Condicional:

```
1 # Filtrado simple
2 df[df['Edad'] > 30]
3
4 # Mltiples condiciones
5 df[(df['Edad'] > 25) & (df['Salario'] > 55000)]
6
7 # Usando isin()
8 df[df['Ciudad'].isin(['Madrid', 'Barcelona'])]
9
10 # Usando query()
11 df.query('Edad > 30 and Salario > 55000')
```

4.5 GroupBy (Agrupamiento)

Ejemplo de Código

Agregación con GroupBy:

```
1 # Agrupar por una columna
2 df.groupby('Ciudad')['Salario'].mean()
3
4 # Mltiples agregaciones
5 df.groupby('Ciudad').agg(
6     Salario_Promedio=('Salario', 'mean'),
7     Salario_Total=('Salario', 'sum'),
8     Empleados=('Nombre', 'count')
9 )
10
11 # Transformación
12 df['Salario_Relativo'] = df.groupby('Ciudad')['Salario'].
13     transform(
14         lambda x: (x - x.mean()) / x.std()
15     )
```

4.6 Merge, Join y Concat

Table 6: Operaciones de Combinación

Operación	Uso	Basado en
<code>merge()</code>	Combinar por columna clave	Columnas
<code>join()</code>	Combinar por índice	Índices
<code>concat()</code>	Apilar DataFrames	Ejes (0 o 1)

Ejemplo de Código

Ejemplos de Combinación:

```

1  # Inner join
2  merged = pd.merge(df1, df2, on='ID')
3
4  # Left join
5  merged = pd.merge(df1, df2, on='ID', how='left')
6
7  # Concatenar verticalmente
8  concatenado = pd.concat([df1, df2], axis=0)
9
10 # Concatenar horizontalmente
11 concatenado = pd.concat([df1, df2], axis=1)

```

5 Visualización de Datos

5.1 Matplotlib - Fundamentos

Ejemplo de Código

Gráficos Básicos con Matplotlib:

```

1 import matplotlib.pyplot as plt
2
3 # Gráfico de líneas
4 plt.figure(figsize=(10, 6))
5 plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
6 plt.xlabel('Eje X')
7 plt.ylabel('Eje Y')
8 plt.title('Mi Gráfico')
9 plt.grid(True)
10 plt.show()
11
12 # Scatter plot
13 plt.scatter(df['Edad'], df['Salario'], alpha=0.6)
14
15 # Histograma
16 plt.hist(df['Salario'], bins=10, edgecolor='black')

```

5.2 Seaborn - Gráficos Estadísticos

Table 7: Gráficos Comunes con Seaborn

Gráfico	Función
Histograma con KDE	<code>sns.histplot(data, kde=True)</code>
Box plot	<code>sns.boxplot(x, y, data)</code>
Violin plot	<code>sns.violinplot(x, y, data)</code>
Heatmap	<code>sns.heatmap(corr, annot=True)</code>
Pair plot	<code>sns.pairplot(df)</code>
Bar plot	<code>sns.barplot(x, y, data)</code>

6 Machine Learning con Scikit-Learn

Definición

Scikit-learn es la librería más popular para machine learning en Python, proporcionando algoritmos de clasificación, regresión, clustering y más.

6.1 Preparación de Datos

Ejemplo de Código

Pipeline de Preparación:

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 # Separar características y target
5 X = df[['Edad', 'Salario']]
6 y = df['Categoría']
7
8 # Dividir train/test
9 X_train, X_test, y_train, y_test = train_test_split(
10     X, y, test_size=0.2, random_state=42
11 )
12
13 # Escalar características
14 scaler = StandardScaler()
15 X_train_scaled = scaler.fit_transform(X_train)
16 X_test_scaled = scaler.transform(X_test)

```

6.2 Modelos de Clasificación

Table 8: Algoritmos de Clasificación

Algoritmo	Clase	Uso
Regresión Logística	LogisticRegression	Línea base rápida
Random Forest	RandomForestClassifier	Robusto, poco tuning
Gradient Boosting	GradientBoostingClassifier	Alta precisión
SVM	SVC	Pequeños datasets

Ejemplo de Código

Entrenamiento y Evaluación:

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.metrics import accuracy_score,
   classification_report
3
4 # Crear y entrenar modelo
5 rf = RandomForestClassifier(n_estimators=100)
6 rf.fit(X_train, y_train)
7
8 # Predecir y evaluar
9 y_pred = rf.predict(X_test)
10 print(f'Accuracy: {accuracy_score(y_test, y_pred)}')
11 print(classification_report(y_test, y_pred))
```

6.3 Validación Cruzada y Grid Search

Ejemplo de Código

Optimización de Hiperparámetros:

```
1 from sklearn.model_selection import cross_val_score,
   GridSearchCV
2
3 # Validación cruzada
4 scores = cross_val_score(RandomForestClassifier(), X, y, cv=5)
5 print(f'CV Accuracy: {scores.mean():.3f}')
6
7 # Grid Search
8 param_grid = {
9     'n_estimators': [50, 100, 200],
10    'max_depth': [None, 10, 20]
11 }
12
13 grid = GridSearchCV(RandomForestClassifier(), param_grid, cv
   =5)
14 grid.fit(X_train, y_train)
15 print(f'Mejores parámetros: {grid.best_params_}')
```

7 Librerías Avanzadas

7.1 XGBoost

Table 9: XGBoost - Gradient Boosting Optimizado

Característica	Descripción
Rendimiento	Hasta 10x más rápido que GBM tradicional
Precisión	Ganador frecuente en competencias Kaggle
Regularización	Previene overfitting con L1/L2
Manejo de Nulos	Manejo automático de valores faltantes

7.2 Plotly - Visualización Interactiva

Ejemplo de Código

Gráficos Interactivos:

```

1 import plotly.express as px
2
3 # Scatter interactivo
4 fig = px.scatter(df, x='Edad', y='Salario',
5                 color='Ciudad', size='Salario')
6 fig.show()
7
8 # Gráfico de líneas
9 fig = px.line(df, x='Fecha', y='Ventas',
10              markers=True, title='Ventas')
11 fig.show()

```

7.3 Streamlit - Apps Web

Ejemplo de Código

Aplicación Web Simple:

```
1 import streamlit as st
2 import pandas as pd
3
4 st.title('Mi App de An lisis')
5
6 df = pd.read_csv('datos.csv')
7
8 if st.checkbox('Mostrar datos'):
9     st.write(df)
10
11 st.line_chart(df['Ventas'])
12
13 valor = st.slider('Selecciona valor', 0, 100, 50)
14 st.write('Valor:', valor)
```

8 Mejores Prácticas

Mejor Práctica

Gestión de Datos:

- Usar copias explícitas: `df.copy()` para evitar `SettingWithCopyWarning`
- Encadenar operaciones con cuidado
- Validar datos antes del análisis

Mejor Práctica

Optimización de Rendimiento:

- Usar operaciones vectorizadas en lugar de loops
- Usar dtypes apropiados (`category`, `int32`)
- Evitar `apply()` cuando hay alternativa vectorizada

Mejor Práctica

Manejo de Errores:

- Manejar errores explícitamente con `try-except`
- Validar datos con `assert`
- Usar logging en lugar de prints

8.1 Estructura de Proyecto Recomendada

Ejemplo de Código

Estructura de Directorios:

```
1 proyecto/  
2     data/  
3         raw/           # Datos originales  
4         processed/     # Datos limpios  
5         external/      # Datos externos  
6     notebooks/        # Jupyter notebooks  
7     src/               # Código fuente  
8         __init__.py  
9         data.py        # Carga y procesamiento  
10        features.py     # Feature engineering  
11        models.py       # Modelos ML  
12    models/            # Modelos entrenados  
13    requirements.txt    # Dependencias  
14    README.md
```

9 Conclusiones

Python se ha establecido como el lenguaje dominante para análisis de datos en 2026. Su combinación de simplicidad, poder y ecosistema rico lo hace indispensable para profesionales de datos.

Información Clave**Puntos Clave para Recordar:**

- **NumPy y Pandas** son fundamentales: 80% del trabajo es preparación de datos
- **Scikit-learn** cubre 90% de problemas clásicos de ML
- **Visualización** es crucial para comunicar insights
- **Proyectos reales** enseñan más que tutoriales
- **Deep Learning** es para imágenes/texto, no para datos tabulares

Recomendación Final: Comience con fundamentos (NumPy, Pandas), progrese a visualización (Matplotlib, Seaborn), y luego avance a machine learning (Scikit-learn, XGBoost). La práctica constante con proyectos reales es esencial.

A Recursos de Aprendizaje

A.1 Cursos Recomendados

Table 10: Plataformas de Aprendizaje

Plataforma	Curso	Costo
Coursera	Python for Everybody (Univ. Michigan)	Gratis/Audit
edX	Data Science MicroMasters (UC San Diego)	\$
DataCamp	Data Scientist with Python	Subscription
Fast.ai	Practical Deep Learning	Gratis
Kaggle Learn	Python, Pandas, ML	Gratis

A.2 Datasets para Practicar

- **Kaggle** – Miles de datasets con notebooks de ejemplo
- **UCI ML Repository** – Datasets clásicos para investigación
- **Google Dataset Search** – Buscador de datasets
- **data.gov** – Datos gubernamentales abiertos
- **scikit-learn** – Datasets incorporados (iris, wine, digits)

B Referencias Bibliográficas

- McKinney, W. (2022). *Python for Data Analysis*. O'Reilly Media.

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- Stack Overflow. (2025). *Developer Survey 2025*.
- Simplilearn. (2026). *Top Python Libraries for Data Science*.
- Aprender21. (2026). *Python para Machine Learning: Guía Completa*.